# From Signals to Sockets: How Networks Work, Explained for C Programmers

Made by AI

***Abstract*—Computer networking is usually taught as a stack of categories to memorize: seven OSI layers, "Layer 2" and "Layer 3" devices, TCP versus UDP, a long list of protocols. This tutorial instead derives the structure of networks from a few physical and engineering facts, and then shows what those facts mean for a programmer writing network code in C. It begins with the channel: a signal of limited bandwidth, disturbed by noise, which can carry bits only at a bounded rate and never without error. It explains why links are shared by switching packets rather than circuits, why protocols are organized in layers, and why the Internet Protocol forms a deliberately minimal "narrow waist" that every network can carry. It then presents the end-to-end argument and the transport protocols built on it, and the Berkeley sockets interface through which C programs reach the network. Finally, it gives canonical forms for network code, each illustrated by a small C program run over the loopback interface: byte order, wire formats, the absence of message boundaries in TCP, partial transfers, the latency cost of small writes, address families, error handling, and untrusted input.**

***Index Terms*—Berkeley sockets, end-to-end argument, Internet Protocol, layering, network programming, TCP, UDP.**

## I. Introduction

A programmer who opens a socket uses, without seeing it, a chain of mechanisms that reaches from a voltage on a wire or a radio wave in the air to a call such as `recv()`. Textbooks describe that chain as a set of layers and protocols to be learned by name, and it is easy to come away knowing the names without knowing why the chain has the shape it has, or what it promises to the program at its top.

This paper is the third in a series that tries to replace memorization with derivation. The first explained the Rainbow Series of trusted-systems guidance, and the second explained C as a portable assembly language over a minimal set of machine operations. Here the subject is the network. The structure of the argument is the same. Physical media diverge widely; engineers introduced categories to describe the differences; a small common core makes it possible to connect them all; a portable programming interface sits on that core; and programs are correct only when written in canonical forms that rely on nothing the core does not promise.

Manuscript prepared November 2026.

Section II begins with the channel and its limits. Section III explains why links are shared by packets. Section IV derives layering, and Section V the role of the Internet Protocol as the network's minimal common core. Section VI presents the end-to-end argument and the transport protocols. Section VII introduces the sockets interface, and Section VIII the canonical forms of network code, each demonstrated by a short C program run on Linux over the loopback interface. Section IX discusses what the paper leaves out, and Section X concludes.

## II. The Channel: Signals, Capacity, and Errors

Every network rests on a physical channel: a pair of copper wires, an optical fibre, or a band of radio spectrum. The channel does not carry bits. It carries a signal, and bits must be encoded into the signal at one end and recovered from it at the other.

### A. Two Limits

Two classic results bound what any channel can do. Nyquist's 1928 analysis of telegraph transmission leads to the result, in its usual modern statement, that a channel of bandwidth $B$ can carry at most $2B$ independent signal values per second [1]. With more distinguishable levels per value, more bits can be carried, but noise limits how many levels can be told apart. Shannon resolved the trade-off in 1948. For a channel of bandwidth $B$ disturbed by white Gaussian noise, the highest rate at which information can be sent with arbitrarily small error is

$$C = B \log_2\left(1 + \frac{S}{N}\right) \text{ bits per second}, \tag{1}$$

where $S/N$ is the ratio of signal power to noise power [2], [3]. A telephone-grade channel of about 3 kHz with a signal-to-noise ratio of 30 dB ($S/N = 1000$) can therefore carry at most about 30 kbit/s, whatever the modem. Faster links are faster because they have more bandwidth, a better signal-to-noise ratio, or both.

### B. What the Physics Means for Software

Three consequences reach all the way up to the program. First, *rate is finite*: a link has a capacity, and sending faster than it only fills queues. Second, *delay is unavoidable*: a signal in optical fibre travels at roughly two-thirds of the speed of light in vacuum, about 5 microseconds per

kilometre [4], so a message across a continent takes tens of milliseconds however fast the link. Third, *errors are inevitable*: Shannon's theorem promises arbitrarily small error, not zero error, and real links occasionally deliver corrupted bits. Every frame therefore carries a check value computed from its contents, on Ethernet a 32-bit cyclic redundancy check [5], and a frame that fails the check is discarded.

Different media make very different trade-offs. Copper is cheap and short-range; fibre has enormous bandwidth and very low error rates; radio is shared by everyone in range and suffers from interference and fading. A network that spans all of them must work with the weakest promises any of them can make.

**Key point.** A channel moves bits at a bounded rate, after a delay, and with occasional errors. Everything above it must be designed around those three facts.

## III. Sharing the Channel: Circuits, Packets, and Frames

A dedicated wire between every pair of computers is impossible, so channels must be shared. The telephone network shared them by *circuit switching*: a path is reserved end to end for the duration of a call, whether or not anyone is speaking. Computer traffic is bursty, so a reserved circuit would sit idle most of the time.

### A. Packet Switching

In the early 1960s Baran proposed a different design for a communications network that could survive damage: break messages into small blocks, and route each block independently through a mesh of nodes [6]. Independently, Davies at the National Physical Laboratory in Britain conceived the same idea in 1965, and in a 1966 proposal gave the blocks the name *packets* [7], [8]. Packet switching shares a link statistically: packets from many conversations take turns, so a link is busy whenever anyone has something to send. The price is that packets may wait in queues, may arrive out of order, and may be dropped when a queue overflows.

### B. Frames on a Shared Medium

Within a single local network the same idea appears as the *frame*. Ethernet, as described by Metcalfe and Boggs in 1976, connected many computers to one shared cable [9]. Each frame carried the address of its destination, every station listened, and a station that detected another transmission waited and tried again. Modern Ethernet replaces the shared cable with switches that forward each frame only toward its destination, but the frame, with its addresses and its check value, remains.

**Key point.** Bursty traffic is best served by sharing links packet by packet. The cost is queueing delay, reordering, and loss, which higher layers must handle.

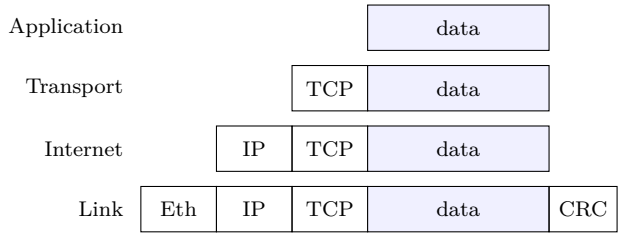



Fig. 1. Encapsulation. Each layer prepends its own header; the link layer also appends a check value. Only the matching layer at the receiver, or a device operating at that layer, reads each header.

## IV. Why the Layers

The jobs described so far are different in kind. Turning bits into signals depends on the medium. Delivering a frame within one network depends on the network's own addressing. Delivering a packet across many networks needs addresses that span them all. Delivering a reliable stream to an application needs sequence numbers and retransmission. Engineers organized these jobs into *layers*, each using the service of the one below and offering a service to the one above.

### A. Encapsulation

Layering is visible in every packet. Each layer adds its own header in front of the data handed down from above, a process called *encapsulation*, as shown in Fig. 1. A web page travels inside a TCP segment, inside an IP packet, inside an Ethernet frame. Each header is read only by the matching layer at the other end, or by intermediate devices that operate at that layer.

### B. Two Models, One Idea

Two layer models are commonly taught. The OSI reference model, standardized by ISO, defines seven layers: physical, data link, network, transport, session, presentation, and application [10], [11]. The Internet's own host requirements describe a simpler stack of link, internet, transport, and application layers [12]. The difference is less important than it seems. The four Internet layers correspond to the OSI data link, network, transport, and application layers, with the physical medium below and OSI's session and presentation functions folded into applications.

The familiar device categories follow directly. A device that forwards frames within one network by their link-layer addresses is a "Layer 2" device, a *switch*. A device that forwards packets between networks by their IP addresses is a "Layer 3" device, a *router*. The numbers are OSI layer numbers, and nothing more: they say which header the device reads.

**Key point.** Layers separate jobs that differ in kind. "Layer 2" and "Layer 3" simply name which header a device reads to make its decision.

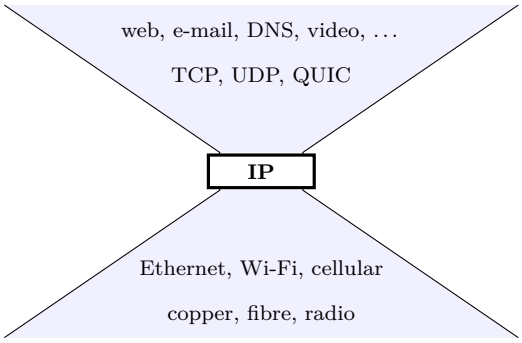



Fig. 2. The protocol hourglass. Diversity above and below a single, deliberately minimal protocol at the waist.

## V. The Narrow Waist: IP as the Common Core

By the early 1970s there were already several incompatible packet networks, built on different media with different frame formats. Cerf and Kahn proposed a way to connect them without changing any of them: a common packet format and addressing scheme, carried over each network as ordinary data, with gateways between networks [13]. That design became the Internet Protocol (IP) and TCP.

### A. What IP Promises

IP's promise is deliberately weak. It offers *best-effort* delivery of individual packets, called datagrams, to an address [14]. A datagram may be lost, duplicated, delayed, or delivered out of order, and IP makes no attempt to prevent it. Each datagram carries a source and destination address, and routers forward it hop by hop toward its destination. Because links have different maximum frame sizes, IP also defines a minimum that every host must handle: 576 bytes for IPv4 [14] and a minimum link size of 1280 bytes for IPv6 [15].

Clark later explained why the promise is so weak. The top goal of the Internet architecture was to interconnect existing networks, and the most important secondary goal was that communication should continue despite the loss of networks or gateways [16]. A minimal service could be implemented over any network at all, and keeping no per-conversation state in the routers meant that a failed router lost nothing that the endpoints could not recover.

### B. The Hourglass

The result is often drawn as an hourglass, Fig. 2: many applications and transport protocols above, many link technologies below, and a single protocol at the narrow waist [17]. The waist is the network's equivalent of the minimal machine of the C paper in this series: the one thing every network agrees to carry, kept small precisely so that everything above and below can vary.

The waist has been widened once. IPv4's 32-bit addresses proved too few, and IPv6 replaced them with 128-bit addresses and a simplified header [15]. The two coexist today, which, as Section VIII shows, affects how portable programs must be written.

**Key point.** IP promises little, so that it can run over anything. Like the minimal machine beneath C, its smallness is the source of its universality.

## VI. End to End: TCP and UDP

If IP may lose, duplicate, and reorder packets, who makes communication reliable? The answer comes from one of the most influential arguments in system design.

### A. The End-to-End Argument

Saltzer, Reed, and Clark observed that a function such as reliable delivery "can completely and correctly be implemented only with the knowledge and help of the application standing at the end points of the communication system" [18]. Checking every link along the way does not remove the need for an end-to-end check, because errors can also occur inside the routers, in the receiving host, or in the application. Implementing reliability in the network is therefore at best a performance optimization. The Internet puts it in the hosts, in the transport layer, and keeps the network simple.

### B. UDP: Almost Nothing Added

The User Datagram Protocol adds very little to IP: an 8-byte header with source and destination ports, a length, and a checksum [19]. Ports let several programs on one host share an address. UDP keeps IP's best-effort service and its message boundaries: each datagram sent is delivered whole or not at all. It suits applications that prefer timeliness to completeness, such as voice and video, or that build their own reliability on top.

### C. TCP: A Reliable Byte Stream

The Transmission Control Protocol builds a reliable, ordered *byte stream* between two endpoints [20]. A connection begins with a three-way handshake that agrees on initial sequence numbers. Every byte is numbered, the receiver acknowledges what has arrived, and the sender retransmits what is not acknowledged in time. *Flow control* keeps the sender from overrunning the receiver's buffer. *Congestion control* keeps senders from overrunning the network. When it was missing, the early Internet suffered a series of "congestion collapses" in which throughput fell by orders of magnitude, on one path from 32 kbit/s to 40 bit/s, until Jacobson introduced the algorithms, based on the principle of "conservation of packets", that TCP still uses [21], [22].

Two quantities explain most of TCP's performance. The *round-trip time* (RTT) is the time for data to reach the receiver and an acknowledgment to return. The *bandwidth-delay product* is the amount of data that must be in flight to keep a link busy: on a 1 Gbit/s path with a 50 ms RTT it is 50 million bits, about 6 megabytes, which is why TCP must keep large amounts of unacknowledged data outstanding, and why a new connection, which starts slowly and probes for capacity, takes several round trips to reach full speed [22], [23].

TABLE I
THE CENTRAL SOCKET CALLS

| Call | Effect |
|---|---|
| `getaddrinfo` | Translate a host name and service into socket addresses |
| `socket` | Create an endpoint of a given family and type |
| `bind` | Attach a local address and port |
| `listen` | Mark a stream socket as accepting connections |
| `accept` | Take the next connection, as a new socket |
| `connect` | Open a connection to a remote address |
| `send`, `recv` | Transfer bytes or datagrams |
| `close` | Release the endpoint |

### D. Newer Transports

The transport layer continues to evolve above the narrow waist. QUIC, standardized in 2021, provides reliable streams over UDP with encryption built in, and avoids having a single lost packet delay every stream on the connection [24]. That it could be deployed at all, without changing a single router, is the hourglass doing its job.

**Key point.** Reliability belongs at the ends. UDP keeps IP's datagrams; TCP turns them into a reliable byte stream with flow and congestion control. Its speed is governed by the round-trip time and the bandwidth-delay product.

## VII. Sockets: The Portable Abstraction

C programs reach the network through the *sockets* interface, introduced in 4.2BSD Unix in 1983 [25] and now part of POSIX [26]. A socket is an endpoint of communication, represented, like an open file, by a small integer called a file descriptor. Its design mirrors the layers below: the program chooses an address family (IPv4 or IPv6), a socket type (a byte stream or datagrams), and a protocol (normally implied by the other two), and the operating system implements everything beneath.

Table I lists the central calls. A server creates a socket, binds it to a local address and port, marks it as listening, and accepts connections, each of which yields a new socket. A client creates a socket and connects it to a server. Both then transfer data with `send()` and `recv()` and finish with `close()`.

The interface is portable in the same sense as C itself. A program written against it runs over Ethernet or Wi-Fi, IPv4 or IPv6, without change, because the abstraction exposes only what the layers below guarantee. That is also its trap: it exposes *exactly* what they guarantee, which is less than many programs assume.

**Key point.** Sockets give C programs a file-like handle on the transport layer. They hide the layers below, but not their limits.

## VIII. Canonical Forms for Network Code

The C paper of this series defined a canonical form as a way of writing a construct that stays inside what is guaranteed, translates predictably, and makes mistakes detectable. Network code needs the same discipline, with one addition: the other end of a connection is a separate program, on a separate machine, possibly hostile, and it may fail at any moment. The forms below follow from one rule, *rely on nothing the layers below do not promise*. Each is illustrated by a small C99 program run on Linux over the loopback interface.

### A. Byte Order

IP and the protocols above it transmit multi-byte integers with "the most significant octet ... transmitted first" [14], the order that the sockets interface calls *network byte order* [26]. Hosts may store integers either way. On the x86-64 machine used here, the address 192.168.0.1, held in a 32-bit integer, is stored in memory as `01 00 a8 c0`; after conversion with `htonl()` it becomes `c0 a8 00 01`, the order required on the wire. The canonical form converts every multi-byte field at the boundary: `htons()` and `htonl()` on the way out, `ntohs()` and `ntohl()` on the way in, or, for fields of any width, the shift-and-OR technique of the C paper, which is correct on every host by construction.

### B. Wire Formats, Not Structures

It is tempting to send a C structure directly with `send(s, &msg, sizeof msg, 0)`. That transmits the compiler's layout, including padding between fields and the host's byte order, neither of which the other end is guaranteed to share. The canonical form defines the wire format byte by byte, writes each field into a byte buffer explicitly, and reads it back the same way. The structure is the program's private representation; the byte sequence is the protocol.

### C. Message Boundaries

TCP delivers a stream of bytes, not a sequence of messages [20]. The following run shows the difference. A client called `send()` three times, with five bytes each time, over TCP and then over UDP:

```
TCP: three send() calls of 5 bytes each
  recv() returned 15 bytes: HELLOWORLDAGAIN
UDP: three sendto() calls of 5 bytes each
  recv() returned 5 bytes: HELLO
  recv() returned 5 bytes: WORLD
  recv() returned 5 bytes: AGAIN
```

Over TCP, the three messages arrived merged into a single receive; on another occasion they might arrive split differently. Over UDP, each datagram arrived separately. A TCP program must therefore mark message boundaries itself. The canonical form is *framing*: precede each message with its length, in network byte order, or end it with a delimiter that cannot appear inside it, and have the receiver read exactly that many bytes.

### D. Partial Transfers

Neither sending nor receiving is guaranteed to transfer as many bytes as requested. POSIX allows a write to a

stream to “write what it can and return the number of bytes written”, and a receive returns whatever data is available [26]. On Linux, a non-blocking TCP socket asked to send 8 MiB at once accepted part of it and then, with its buffer full, refused more:

```
call 1: asked 8388608 bytes, send() accepted 3919467
call 2: send() = -1, errno = EAGAIN (buffer full)
```

Blocking sockets can also return early, for example when interrupted by a signal. The canonical form wraps every transfer in a loop that continues until the whole buffer is done, retries on `EINTR`, and treats a zero return from `recv()` as the peer’s orderly shutdown rather than as data [26]:

```
int send_all(int s, const void *buf, size_t n) {
  const char *p = buf;
  while (n > 0) {
    ssize_t k = send(s, p, n, 0);
    if (k < 0) { if (errno == EINTR) continue;
                 return -1; }
    p += k; n -= (size_t)k;
  }
  return 0;
}
```

### E. Small Writes and Latency

TCP tries to avoid sending many tiny packets. Nagle’s algorithm holds back a small segment while earlier data is still unacknowledged [27], and receivers commonly delay acknowledgments in the hope of combining them with a reply. Together they can stall a common pattern: a client writes a request in two pieces and then waits for the answer. Fifty such exchanges over loopback gave:

```
two writes, Nagle on (default)    43.105 ms per request
two writes, TCP_NODELAY on         0.059 ms per request
one write (header+body together)   0.059 ms per request
```

The two-write pattern was about 700 times slower, because each second write waited for a delayed acknowledgment. Disabling Nagle’s algorithm with the `TCP_NODELAY` option [28] removed the delay, but so did the simpler fix. The canonical form assembles each complete message in a buffer and writes it with one call, which is both faster and independent of socket options.

### F. Names and Address Families

A host name may correspond to IPv4 addresses, IPv6 addresses, or both, and a portable program must not assume either. The canonical form resolves names with `getaddrinfo()` [29], requesting `AF_UNSPEC`, and tries each returned address in turn until one works. Asked for the addresses a server should listen on, the system used here returned both families:

```
IPv4  0.0.0.0
IPv6  ::
```

Clients that try addresses one after another can be slow when the first family is broken, so modern implementations attempt both families in parallel, an approach known as Happy Eyeballs [30].

### G. Errors, Timeouts, and Shutdown

Every socket call can fail, and network failures are normal events, not exceptional ones. The canonical form checks every return value. It sets a timeout on every operation that waits for the peer, with `poll()` or the `SO_RCVTIMEO` option [31], because a peer that vanishes without closing the connection otherwise leaves the program waiting indefinitely. And it prevents the signal `SIGPIPE`, which by default terminates a process that writes to a connection the peer has closed [26], by ignoring the signal or passing the `MSG_NOSIGNAL` flag, part of POSIX since 2008, so that the error is reported as a return value instead.

### H. Untrusted Input

Everything received from the network is untrusted. A length field read from a message is only a claim, and copying that many bytes into a fixed buffer without checking is one of the oldest ways to compromise a program. The canonical form validates every field against the protocol’s limits before using it, and rejects a message that fails. This also tempers an old maxim. Postel’s *robustness principle*, “be conservative in what you do, be liberal in what you accept from others” [32], helped early implementations interoperate, but tolerating malformed input lets errors spread and invites attack, and the Internet Architecture Board now recommends that protocols be designed and maintained with strict, well-defined behavior instead [33]. Where confidentiality or integrity matters, the canonical form uses an established secure transport such as TLS 1.3 [34] rather than inventing its own.

**Key point.** Rely on nothing the layers below do not promise: convert byte order, define wire formats explicitly, frame messages, loop over partial transfers, write whole messages at once, resolve names for both families, check every result, time out every wait, and validate every input.

## IX. Discussion

The paper has necessarily left much out. Routing, the protocols by which routers learn where to send packets, deserves a paper of its own, as do the name system, wireless links, and the event-driven interfaces such as `poll()` and its successors that servers use to handle many connections at once. The intent has been to show the structure that makes those topics fit together, not to survey them.

The structure is the same one found beneath C. Physical media diverge as processors do, each with its own capacity, delay, and error behavior. Engineers named the differences with layer numbers and device categories, much as they named processor differences with CISC, RISC, and cache levels. A deliberately minimal common core, IP, makes interconnection possible, as the minimal machine makes portable C possible. A portable interface, sockets, sits on that core, and correct programs are written in canonical forms that rely only on what the core promises. In both cases the categories need not be memorized; they follow from the structure.

TABLE II
SUMMARY OF CANONICAL FORMS FOR NETWORK CODE

| Concern | What the network does not promise | Canonical form |
|---|---|---|
| Byte order | That the peer stores integers as this host does | Convert every multi-byte field at the boundary (`htonl`/`ntohl` or shifts and ORs) |
| Wire format | That the peer shares the compiler's layout | Define the format byte by byte; never send a structure directly |
| Boundaries | That TCP preserves message boundaries | Length-prefix or delimiter framing; read exactly the framed length |
| Partial transfer | That a call moves all requested bytes | Loop until complete; retry on `EINTR`; treat `recv() == 0` as shutdown |
| Small writes | That separate writes leave promptly | Assemble each message and send it with one call; use `TCP_NODELAY` only when needed |
| Addresses | That a host has one address family | `getaddrinfo` with `AF_UNSPEC`; try every result |
| Failures | That the peer stays reachable | Check every result; time out every wait; avoid `SIGPIPE` |
| Input | That the peer is honest or correct | Validate every field against protocol limits; reject malformed messages |
| Security | That anyone on the path is trustworthy | Use an established secure transport such as TLS |

## X. Conclusion

A network is a physical channel with finite rate, unavoidable delay, and occasional errors, shared packet by packet, organized in layers, and joined across all its varieties by one deliberately minimal protocol. Reliability is added at the ends, where the end-to-end argument says it must be, and the sockets interface gives C programs a portable handle on the result. That handle exposes exactly what the network promises, and no more. Programs that respect this, by converting byte order, framing messages, looping over partial transfers, writing whole messages, handling every address family and every failure, and trusting no input, are correct on every network. Those that do not may work in testing and fail in the field.

## References


[1] H. Nyquist, "Certain topics in telegraph transmission theory," *Transactions of the American Institute of Electrical Engineers*, vol. 47, no. 2, pp. 617–644, Apr. 1928.

[2] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, Jul. 1948.

[3] ——, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, no. 4, pp. 623–656, Oct. 1948.

[4] ITU-T, "One-way transmission time," ITU-T Recommendation G.114, May 2003. [Online]. Available: https://www.itu.int/rec/T-REC-G.114-200305-I/en

[5] IEEE, "IEEE standard for Ethernet," IEEE Std 802.3-2022 (Revision of IEEE Std 802.3-2018), Jul. 2022. [Online]. Available: https://ieeexplore.ieee.org/document/9844436

[6] P. Baran, "On distributed communications: I. introduction to distributed communications networks," RAND Corporation, Santa Monica, CA, USA, Research Memorandum RM-3420-PR, Aug. 1964. [Online]. Available: https://www.rand.org/pubs/research_memoranda/RM3420.html

[7] D. W. Davies, "Proposal for a digital communication network," National Physical Laboratory, Tech. Rep., Jun. 1966. [Online]. Available: https://www.computerhistory.org/collections/catalog/102660191

[8] ——, "An historical study of the beginnings of packet switching," *The Computer Journal*, vol. 44, no. 3, pp. 152–162, 2001.

[9] R. M. Metcalfe and D. R. Boggs, "Ethernet: Distributed packet switching for local computer networks," *Communications of the ACM*, vol. 19, no. 7, pp. 395–404, Jul. 1976.

[10] H. Zimmermann, "OSI reference model—The ISO model of architecture for open systems interconnection," *IEEE Transactions on Communications*, vol. 28, no. 4, pp. 425–432, Apr. 1980.

[11] ISO/IEC, "Information technology—Open systems interconnection—Basic reference model: The basic model," International Standard ISO/IEC 7498-1:1994, Nov. 1994. [Online]. Available: https://www.iso.org/standard/20269.html

[12] R. Braden, "Requirements for Internet hosts – communication layers," RFC 1122, IETF, RFC 1122, Oct. 1989. [Online]. Available: https://www.rfc-editor.org/rfc/rfc1122

[13] V. G. Cerf and R. E. Kahn, "A protocol for packet network intercommunication," *IEEE Transactions on Communications*, vol. 22, no. 5, pp. 637–648, May 1974.

[14] J. Postel, "Internet protocol," RFC 791, IETF, RFC 791, Sep. 1981. [Online]. Available: https://www.rfc-editor.org/rfc/rfc791

[15] S. Deering and R. Hinden, "Internet protocol, version 6 (IPv6) specification," RFC 8200 (STD 86), IETF, RFC 8200, Jul. 2017. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8200

[16] D. D. Clark, "The design philosophy of the DARPA internet protocols," in *Proceedings of the ACM SIGCOMM '88 Symposium on Communications Architectures and Protocols*, Stanford, CA, USA, Aug. 1988, pp. 106–114.

[17] S. Deering, "Watching the waist of the protocol hourglass," Plenary talk, 51st IETF Meeting, London, U.K., Aug. 2001. [Online]. Available: https://www.ietf.org/proceedings/51/slides/plenary-1/index.html

[18] J. H. Saltzer, D. P. Reed, and D. D. Clark, "End-to-end arguments in system design," *ACM Transactions on Computer Systems*, vol. 2, no. 4, pp. 277–288, Nov. 1984.

[19] J. Postel, "User datagram protocol," RFC 768, IETF, RFC 768, Aug. 1980. [Online]. Available: https://www.rfc-editor.org/rfc/rfc768

[20] W. Eddy, "Transmission control protocol (TCP)," RFC 9293 (STD 7), IETF, RFC 9293, Aug. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9293

[21] V. Jacobson, "Congestion avoidance and control," in *Proceedings of the ACM SIGCOMM '88 Symposium on Communications Architectures and Protocols*, Stanford, CA, USA, Aug. 1988, pp. 314–329.

[22] M. Allman, V. Paxson, and E. Blanton, "TCP congestion control," RFC 5681, IETF, RFC 5681, Sep. 2009. [Online]. Available: https://www.rfc-editor.org/rfc/rfc5681

[23] I. Grigorik, *High Performance Browser Networking*. O'Reilly Media, 2013. [Online]. Available: https://hpbn.co/

[24] J. Iyengar and M. Thomson, "QUIC: A UDP-based multiplexed and secure transport," RFC 9000, IETF, RFC 9000, May 2021. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9000

[25] S. J. Leffler, R. S. Fabry, and W. N. Joy, "A 4.2BSD interprocess communication primer," Computer Science Division, University of California, Berkeley, Berkeley, CA, USA, Tech. Rep. UCB/CSD-83-145, Jul. 1983, draft of July 27, 1983. [Online]. Available: https://www2.eecs.berkeley.edu/Pubs/TechRpts/1983/5451.html

[26] IEEE and The Open Group, "IEEE/Open Group standard for information technology–portable operating system interface (POSIX) base specifications, issue 8," IEEE Std 1003.1-2024 / The Open Group Base Specifications Issue 8, 2024. [Online]. Available: https://pubs.opengroup.org/onlinepubs/9799919799/

[27] J. Nagle, "Congestion control in IP/TCP internetworks," RFC 896, IETF, RFC 896, Jan. 1984. [Online]. Available: https://www.rfc-editor.org/rfc/rfc896

[28] Linux man-pages project, "tcp(7) – TCP protocol," Linux Programmer's Manual, man-pages 6.19, 2026. [Online]. Available: https://man7.org/linux/man-pages/man7/tcp.7.html
[29] R. Gilligan, S. Thomson, J. Bound, J. McCann, and W. Stevens, "Basic socket interface extensions for IPv6," RFC 3493, IETF, RFC 3493, Feb. 2003. [Online]. Available: https://www.rfc-editor.org/rfc/rfc3493
[30] D. Schinazi and T. Pauly, "Happy eyeballs version 2: Better connectivity using concurrency," RFC 8305, IETF, RFC 8305, Dec. 2017. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8305
[31] Linux man-pages project, "socket(7) – Linux socket interface," Linux Programmer's Manual, man-pages 6.19, 2026. [Online]. Available: https://man7.org/linux/man-pages/man7/socket.7.html
[32] J. Postel, "DoD standard transmission control protocol," RFC 761, IETF, RFC 761, Jan. 1980. [Online]. Available: https://www.rfc-editor.org/rfc/rfc761
[33] M. Thomson and D. Schinazi, "Maintaining robust protocols," RFC 9413, Internet Architecture Board (IAB), RFC 9413, Jun. 2023. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9413
[34] E. Rescorla, "The transport layer security (TLS) protocol version 1.3," RFC 8446, IETF, RFC 8446, Aug. 2018. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8446